

Exact List-Edge Labelings on Finite Bipartite Multigraphs

Standardized research and formalization report

Dylan Tague

Independent Capstone Research Project

September 11, 2026

Abstract

This report presents a consolidated account of the exact list-edge labeling project. It replaces exploratory versions and presentation-specific drafts with one standardized statement of the definitions, proof architecture, verified consequences, finite certificates, formalization status, and open research questions.

The central object is a finite bipartite multigraph whose parallel edges are distinct edge copies. Every left vertex is assigned a finite set of labels of cardinality equal to its degree. An admissible labeling uses exactly the assigned labels at each left vertex and is proper at every right vertex. The main comparison is between the number of admissible exact-list labelings and the number of ordinary proper edge-colorings from a fixed palette.

The principal inequality is

$$N(L,R) \geq c_k(R),$$

when every left vertex has degree k . The audited mathematical argument uses indexed support families, union/intersection uncrossing, a common residual fiber, a strictly increasing finite potential, and a nested terminal form. That argument is coherent as an ordinary manuscript proof, but the complete fiber comparison and termination chain have not yet been translated into a kernel-checked Lean theorem. The current Lean project formalizes the finite data model and elementary support lemmas and records the remaining proof obligations without introducing them as axioms.

The report distinguishes four kinds of status:

1. **Lean-checked:** a declaration has been accepted by the Lean kernel.

2. **Manuscript-verified:** a mathematical argument has been audited, but its

complete formal translation has not been kernel-checked.

3. **Finite-certified:** an explicitly defined finite object or computation has been independently reproduced. This is evidence about that finite object, not a proof of a universal theorem.

4. **Open or incomplete:** the archive contains no complete proof or validated

counterexample for the stated universal claim.

These distinctions are part of the result. A formalization project is useful only if it says exactly which propositions have been proved and which remain research obligations.

1. The problem and its scope

Let

$$R=(X,Y;E,l,r)$$

be a finite bipartite multigraph. Here X and Y are the left and right vertex sets, E is the type of edge copies, and

$$l:E \rightarrow X, r:E \rightarrow Y$$

are the endpoint maps. Two parallel edges are different elements of E and therefore remain different positions in every count. Degrees count edge copies.

Let Λ be a finite label alphabet. An exact left-list assignment is a family

$$L=(L_x)_{x \in X}, L_x \subseteq \Lambda,$$

with

$$|L_x| = R(x)$$

for every x . A labeling $\sigma: E \rightarrow \Lambda$ is admissible when:

1. the labels on the edges incident with x are exactly L_x , each used once; and
2. labels on distinct edges incident with the same right vertex are distinct.

Write $N(L, R)$ for the number of admissible labelings.

When every left degree is k , let $c_k(R)$ denote the number of proper edge colorings with the fixed labeled palette $\text{Fin } k$. On a left- k -regular graph, properness at a left vertex forces all k palette labels to appear there. The comparison therefore has the same exact-use convention on both sides.

What is not being claimed

This report does not identify the exact-left-list problem with arbitrary per-edge list coloring. In the arbitrary per-edge model, the list attached to one edge may differ from the list attached to another edge at the same left vertex. That is a different counting problem. Existence theorems for list edge-coloring do not by themselves imply the count comparison above.

The project also does not claim a solution for arbitrary matroids. Its cleanest matroid interpretation is the purely parallel specialization, where the incidence graph gives a faithful bipartite graph model.

2. Supports and the exact-list encoding

For a label c , define its indexed support by

$$S_c = \{x \in X : c \in L_x\}.$$

The family $(S_c)_{(c \in \Lambda)}$ is indexed by labels. Equal supports for two different labels are not identified, because the two labels remain distinct choices in every labeling count. This convention is essential for the uncrossing and counting arguments.

Conversely, an indexed support family $S: \Lambda \rightarrow \text{Finset } X$ determines left lists by

$$L_S(x) = \{c \in \Lambda : x \in S_c\}.$$

The exactness condition is

$$|L_S(x)| = R(x)$$

for every left vertex. The Lean development contains both directions of this finite representation and keeps the label index visible.

Empty supports can be treated as empty slots or removed after the corresponding invariance has been proved. They must not be silently discarded before the label indexing convention is fixed.

3. The uncrossing mechanism

Take two distinct labels a and b with supports $A = S_a$ and $B = S_b$. The uncrossed family replaces them by

$$A \cup B, \text{ and } A \cap B,$$

and leaves every other indexed support unchanged. At each left vertex x ,

$$1_{(x \in A \cup B)} + 1_{(x \in A \cap B)} = 1_{(x \in A)} + 1_{(x \in B)}.$$

Thus the number of labels assigned to every left vertex is preserved. The operation is therefore compatible with exact lists whenever the original family was exact.

3.1 The common residual fiber

Freeze all labels other than a and b . A residual placement records the unaffected labels subject to their support incidences, edge-disjointness, and right-side distinctness. The fiber domain must contain every placement that satisfies those unaffected constraints, whether or not it extends to a full labeling on the original or uncrossed support family.

This common domain is important. Restricting the domain to residual placements that already extend on one side would make the comparison circular and could silently omit placements created by uncrossing.

For a fixed residual placement, the remaining two-label completions decompose into alternating paths and cycles. The endpoint behavior of a path determines which of the two labels can be assigned at its ends. The union/intersection replacement preserves the total number of completions after the path and cycle contributions are compared. Summing over the common residual fibers gives the local inequality

$$N(S) \leq N(\text{uncross}(S, a, b)).$$

The audited source proof is careful about parallel edge copies: the two particular edge positions in a residual path must be named, not identified merely by their endpoints.

3.2 A finite potential

Define

$$\Phi(S) = \sum_{c \in \Lambda} |S_c|^2.$$

For two incomparable finite sets A and B ,

$$(|A \cup B|^2 + |A \cap B|^2) - (|A|^2 + |B|^2) = 2|A \setminus B| |B \setminus A| > 0.$$

Therefore every uncrossing of an incomparable pair strictly increases Φ , while Φ ranges over a finite set of integer values. Repeated uncrossing

terminates at a family in which the nonempty supports are nested. Since every step is nondecreasing for the admissible-labeling count, a nested family gives the terminal comparison needed for the lower bound.

The Lean project defines the support potential, incomparability, the union/intersection operation, and the proposition expressing the potential increase. The cardinal identity and the residual-fiber comparison remain the main formal proof obligations.

4. The exact-list lower bound

Theorem A - exact-list lower bound

For every finite bipartite multigraph R whose left vertices have degree k , and every exact left-list assignment L with $|L_x|=k$,

$$N(L,R) \geq c_k(R).$$

Status

Manuscript-verified; Lean formalization in progress.

The audited proof chain has the following logical dependencies:

1. exact left-list assignments are represented by indexed supports;
2. union/intersection uncrossing preserves every left-list cardinality;
3. the two-label residual completion count is nondecreasing under uncrossing;
4. the support potential forces finite termination;
5. the nested terminal family has at least the ordinary palette count.

The first three items must be stated with a common residual fiber and explicit edge-copy conventions. The final formal theorem should not be declared until all five links are represented by Lean terms. The current Lean proposition `ExactListLowerBound` records the intended universal statement; it is not a placeholder proof and it is not currently asserted as a theorem.

Consequences in the same model

The lower bound yields the ordinary existence of an admissible exact-list labeling whenever the ordinary coloring count is positive. It also provides strictness criteria when a residual uncrossing step has a crossing witness. These consequences remain one-sided exact-list results. They do not extend automatically to arbitrary per-edge lists.

5. Equality and the common core

The common core of an exact list assignment is

$$C = \{x \in X \mid L_x \neq \emptyset\},$$

the set of labels whose supports equal all of X . In a connected regular instance, the natural equality question asks when

$$N(L, R) = c_k(R).$$

The audited research reduces the converse direction of the common-core criterion to a three-part two-factor property, denoted TF_k in the source notation. The reduction is meaningful because full common labels can be stripped and the residual count decomposes into finite sums indexed by matching or factor choices. Equality of the total sum forces equality of the relevant residual terms under the stated regularity hypotheses.

The TF_k condition

For a connected k -regular bipartite multigraph and a partition

$$X = A \sqcup B \sqcup C$$

into three nonempty parts, TF_k asks for a spanning 2-factor with at least one component meeting all three parts. It does not require the factor to be Hamiltonian, and it does not require every component to meet all three parts. Those stronger formulations are different statements.

Status of equality claims

- The common-core framework and its reduction to TF_k are coherent at the manuscript level under the stated regularity and incidence hypotheses.
- The converse that every equality case has a common core of size at least $k-1$ is open in the generality under discussion.
- The universal TF_k statement is open.
- A surplus formula that omits the necessary right-degree guard is false and is not included as a Lean theorem.
- Historical claims about a unique identity minimizer are not used; the audit found counterexamples to that formulation.

6. Cubic structure: EEP, EVP, two-factors, and Pfaffian branches

The cubic specialization studies prescribed matching and factor properties.

- **EEP** prescribes two edge copies and asks for a compatible complementary structure.
- **EVP** prescribes an edge copy and a nonincident vertex.
- **TF₃** is the cubic instance of the three-part 2-factor condition.

The precise matching convention matters. A perfect matching saturates both shores; a left-saturating matching need not. A complementary component must be defined using the matched port, not merely an entry port. Parallel edges remain distinct in all prescribed-pair statements.

Port-mask intersection

When two cubic factors are glued across three edge-copy ports, the relevant cross-shore condition becomes an intersection condition on port masks. If one factor supplies a mask S and the other supplies a mask T , a gluing is possible exactly when

$$S \cap {}^{(-1)}(T)$$

for the port identification σ , provided the factors satisfy the required nonemptiness and matching hypotheses. This is a useful reduction, not a universal theorem about all cubic graphs.

Pfaffian subclass

The Pfaffian branch combines square-switch arguments, prescribed-pair properties, brace structure, and external matching theorems. The audited argument is a coherent manuscript-level route for the stated Pfaffian subclass when those external inputs are supplied with their exact hypotheses. Several technical boundaries remain explicit:

1. a square-switch argument must match the correct degree and endpoint data;
2. a pure-interface lower bound needs endpoint hypotheses;
3. tight-cut contraction must preserve the exact port convention;
4. the external brace and matching theorems are not yet formalized in Lean.

Thus the Pfaffian branch is not silently promoted to a kernel-checked theorem, and it does not settle universal TF_3 .

Proposed counterexamples

The archive contains proposed cubic obstructions and conditional reductions, but no validated graph-and-certificate witness for a universal TF_3 failure. Claims based only on search totals, unsupported gadget sketches, or missing graph files are not mathematical counterexamples. They remain research notes, not theorem inputs.

7. The purely parallel and Latin-square specialization

In a purely parallel matroid presentation, the ground set is partitioned into parallel classes. A row-respecting decomposition chooses one element from each class in every row and requires each output column to be a basis. The class-row incidence structure is a bipartite graph, and valid decompositions correspond to proper edge-colorings in the exact-list model.

For the complete bipartite graph $K_{(n,n)}$, the ordinary proper n -edge colorings are Latin squares of order n , with labels and rows/columns kept distinct according to the counting convention. The exact-list lower bound therefore gives the purely parallel Latin-floor consequence once Theorem A is formally completed.

The full Latin-floor statement for arbitrary matroids is not established by this graph reduction. The reduction preserves a special class; it does not turn a purely parallel proof into a theorem for all matroid presentations. The audit also separates strict growth from nonnegativity: a nonnegative difference does not imply strict positivity.

8. Finite certificates and computational evidence

Finite enumeration is retained as reproducibility evidence and regression testing. It is never imported as an axiom for an infinite theorem.

8.1 Explicit six-by-six graph

The independently checked graph has bipartite adjacency matrix

[1 1 1 0 0 0 \ 1 1 0 1 0 0 \ 1 1 0 0 1 0 \ 0 0 1 1 0 1 \ 0 0 1 0 1 1 \ 0 0 0 1 1 1].

It has 18 perfect matchings. After deleting the selected edge (1,4), 12 remain. For the selected nonincident vertex 12, 8 of those are witnesses, so the fraction is

$$(8)/(12)=23.$$

The minimum over the tested nonincident edge-vertex pairs is also $2/3$, and the independent checker found no failure for this graph. This corrects the earlier $1/17$ statistic; the corrected number is the result used by the formalization record.

8.2 Bounded enumeration

The independent all-labelled cubic checks report the following counts for the small matrix domains:

left order	all matrices	connected matrices	edge-copy perfect matchings
2	4	2	10
3	55	31	258
4	2,008	1,272	17,064

The same checking program records the relevant witness predicates and found no small failure in its bounded domain. Fifteen explicit Latin witnesses and a complete reduced-Latin census at order seven are retained as finite artifacts. Additional multigraph and simple-graph enumerations were checked over the bounded search domains recorded during the research process. Their status is summarized here rather than published as separate exploratory reports.

8.3 Computational limitations

Several archived claims cannot be promoted to certificates:

- an old C++ run has a buffer-overflow issue;
- some advertised large searches depend on missing /mnt/data modules
or

absent graph corpora;

- an 84 value is a minimum surplus over a restricted 122-system domain, not

a universal minimum count;

- unsupported search totals without matrices, seeds, logs, or executable provenance remain narrative claims;
- finite evidence does not cross the Lean kernel trust boundary.

The finite checks and their limitations are summarized in this report. A finite result is used only for the explicitly defined object and search domain; it is not treated as an opaque premise for a universal theorem.

9. Lean formalization

The public Lean project is `BachThesisLean`. It uses Lean 4, Lake, and Mathlib, with the declared toolchain stored in `lean-toolchain`.

9.1 Current modules

`Basic/Multigraph.lean` defines finite edge-copy bipartite multigraphs, incident edge sets, degrees, and regularity predicates.

`Basic/ExactLists.lean` defines exact left-list assignments, labels used at a left or right vertex, admissibility, finite admissible-labeling sets, their cardinality, and the uniform palette count.

`Uncrossing/Support.lean` defines indexed support families, the conversion between lists and supports, union/intersection uncrossing, support potential, incomparability, and the exact support-incidence proposition. It also proves the elementary list/support inverse statements used by later files.

Conjectures/Status.lean defines the main lower-bound proposition and records the audited status of the equality, cubic, and finite-certificate claims. These status entries are documentation structures, not mathematical axioms.

9.2 What Lean currently verifies

The current source milestone contains kernel-checkable declarations for:

- the finite graph and edge-copy data model;
- membership characterizations for left and right incident edge sets;
- degree and regularity definitions;
- exact-list data and finite admissibility/count definitions;
- uniform palette assignments;
- list/support conversion and its elementary inverse identity;
- the finite union/intersection support operation;
- elementary support lemmas away from the two uncrossed labels.

The source contains no user-defined axiom, sorry, or admit used to assert the research conclusions. A proposition declaration is not counted as a proof: ExactListLowerBound and PotentialIncreaseStatement identify targets whose proof terms still have to be supplied.

9.3 Remaining formal obligations

The central Lean work is to formalize, in order:

1. exact finite residual-fiber objects;
2. the path/cycle completion-count comparison;

3. preservation of exactness and admissibility under support uncrossing;
4. strict potential increase for incomparable supports;
5. finite termination and the nested support normal form;
6. the comparison between the nested terminal count and $c_k(R)$;
7. equality reductions, cubic factors, and external matching inputs where desired.

The main theorem should be marked complete only after these components compile and their declarations have explicit proofs. A successful CI build verifies the source that is present; it does not automatically prove propositions that remain unimplemented.

10. Complete claim-status table

Claim or object	Public status	Formalization status
Finite bipartite multigraph with distinct edge copies	established definition	Lean source milestone
Exact left-list assignment and admissible count	established definition	Lean source milestone
Indexed list/support conversion	elementary theorem	Lean-checked source milestone
Union/intersection cardinal preservation	elementary finite identity	formal target; completion tied to exact incidence lemmas
Exact-list lower bound $N(L,R) \geq c_k(R)$	coherent audited proof	main Lean theorem pending
Nested terminal support normal form	audited proof architecture	Lean proof pending
Purely parallel Latin-floor consequence	conditional on the main theorem	formal proof pending
Common-core equality converse	open research problem	not an axiom
Reduction of common-core equality to TF_k	audited reduction	Lean translation pending
Universal TF_k	open research problem	not an axiom

Claim or object	Public status	Formalization status
Universal cubic TF_3	open research problem	not an axiom
Pfaffian cubic subclass route	manuscript argument with external inputs	not Lean-checked
Arbitrary per-edge-list count comparison	separate and unresolved	outside current theorem
Explicit six-by-six graph statistic	18, 12, 8, fraction $2/3$	finite-certified
Bounded cubic enumerations	exact within stated domains	finite-certified
Proposed universal cubic disproof	no validated certificate	excluded from theorem layer
General matroid Latin-floor claim	not established by graph specialization	open

11. Reproducibility and public presentation

The public repository contains the Lean source, this standardized report, the formalization report, build-status notes, and the audit/evidence files needed to understand the claim boundaries. Exploratory presentation drafts and private source manuscripts are not part of the canonical public document set.

From the repository root, the intended build is:

```
``text lake update lake build ``
```

The GitHub workflow installs the declared Lean toolchain, resolves the Lake dependencies, and runs the build in a clean environment. Finite computation results are stored as explicit data and checked by their accompanying programs; they are not used as opaque premises in Lean.

The research process distinguishes what belongs in the formalization from what must remain a conjecture or research note. The final public presentation therefore makes a narrower but stronger claim: every result is labeled according to the kind of verification it has actually received.

12. Conclusion

The project establishes a precise and reusable graph model for exact left-list edge labelings, identifies a compelling uncrossing route to the ordinary coloring lower bound, and supplies finite evidence for several cubic and Latin specializations. The research is substantial, but its strongest universal conclusions are not all formalized yet.

The correct mathematical endpoint of this report is consequently not a claim that every conjecture is solved. It is a complete map from the original counting problem to its verified definitions, audited proof chain, finite certificates, formal proof obligations, and open frontiers. That map is the foundation for completing the Lean proof without silently upgrading evidence into theoremhood.